XGBoost Regressor – Yellow Taxi Fares

IEOR 4526
Rosy Zhang (zz3276)

Project Overview

This project aims to develop a predictive model for NYC Yellow Taxi total fare charges using TLC trip records from January to October 2025. The process begins with data acquisition and filtering, where only Yellow Taxi trips are retained to ensure a consistent fare structure, and data prior to 2025 is excluded due to the introduction of the new CBD congestion fee. After obtaining the raw dataset, data cleaning and feature preparation are performed using PySpark, including removing invalid entries, constructing temporal and trip-based variables, and enriching the dataset with location-based features such as borough and service zones. Exploratory data analysis is then conducted to understand overall distributions, detect outliers, and examine relationships between key predictors and fare outcomes. Categorical features are also transformed into vectors with one-hot encoding as the last step of data preparation. With a refined feature set, the modeling stage involves training an XGBoost regression model on Vertex AI, leveraging scalable cloud infrastructure to handle large data efficiently. Model performance is evaluated using RMSE, and Coefficient of Determination, and feature importance is analyzed with the trained model to interpret the primary features of fare variation.

Data

This project uses the NYC TLC Yellow Taxi Trip Records spanning January 2025 to October 2025. The primary goal of this project is to predict the total fare charged to customers using relevant trip features. This dataset was selected due to following reasons:

1. Yellow taxi trips have consistent fare composition and detailed trip attributes that can be converted to features for prediction.
2. Different trip categories (Yellow, Green, FHV, FHV High Volume) follow different charging algorithms and trip patterns. Mixing them introduces unnecessary variation. Restricting the dataset to Yellow Taxi trips only ensures the model captures a coherent and interpretable relationship between predictors and fare outcomes.

3. On January 5, 2025, a new surcharge of `cbd_congestion_fee` was introduced to the fare composition, which could increase the total fare charge to customers. To ensure that the model reflects a consistent pricing pattern, all data prior to 2025 were excluded.

The required data files are provided in parquet format on the NYC TLC public data website. Using an automated request to filter and retrieve only the 2025 Yellow Taxi parquet file URLs, a complete list of download links was generated programmatically. These files were then transferred into a GSC bucket by executing download and upload commands within Google Cloud Shell.

Data Cleaning and Feature Preparation

Cleaning the TLC Dataset

1. **Credit Card Payment Trip Filtering and Valid Record Checks**
   First, we ensured only records with non negative fare compositions are being kept in the dataset, and any missing fare values are filled with zero.
   Only credit card payment trips (`payment_type = 1`) are retained, as they tend to be more reliable and contain complete tip information. Trips with invalid monetary values, such as non-positive `fare_amount`, `total_amount`, or non-positive `trip_distance`, are removed.
   Additionally, any records missing essential fields (timestamps or location IDs) are excluded.

2. **Trip Duration Calculation and Extreme Value Removal**
   Trip duration in seconds is computed as the difference between the dropoff and pickup timestamps. Trips that are too short (less than 5 minutes) and too long (longer than 15 hours) are likely data entry errors and are filtered out.

3. **Reconstruction of Total Fare Charges**
   Through inspection of the dataset, I noticed that the add-up value of all fare columns are more than the value recorded in the `total_amount` column. The difference was likely due to the `cbd_congestion_fee` was not included in the `total_amount` calculation in the original dataset. In order to capture the real cost of trips, a new field,

`computed_total_charge`, is created by summing up all charge fields. This new computed field will be the target of prediction.

4. **Time-Based Features Extraction**
   Several time-based features are derived from the pickup timestamp to capture temporal patterns in taxi demand and pricing:
   - Extracts pickup hour, day of week, and month
   - Cyclic transformations of pickup hour to model periodic behaviors
   - Adds a rush hour indicator to identify weekday morning and evening peak travel periods

5. **Average Speed Calculation**
   Average trip speed (mph) is computed using the reported trip distance and duration. This variable is used both as a predictive feature and as a quality-control measure to identify unrealistic combinations of distance and travel time.
   According to NYC.gov, the maximum legal speed limit in New York City is 50 mph; therefore, only trips with an average speed between 0 and 55 mph are retained to allow for minor deviations and measurement noise. This filtering step removes records with implausible speeds that likely result from erroneous timestamps or meter reporting.

Joining With Zone Lookup Table

The Taxi Zone lookup file contains mappings between location IDs and their corresponding borough, zone name, and service zone classifications. Joining was performed twice on `PU_LocationID` and `DO_Location_ID`. After the join, each trip record is enriched with pick up and drop off borough, zone name, and service zone information.
Additional engineered features were created to summarize spatial relationships between the origin and destination.

- A binary indicator (`same_zone`) identifying whether the pickup and drop-off occurred in the same taxi zone. Same zone trips tend to be shorter and might indicate less charge.
- `borough_pair` describing the directional flow between boroughs (e.g., "*Manhattan_to_Brooklyn*")

These zone-based features help the model learn meaningful geographic distinctions, such as differences in trip length, congestion levels, and fare amounts across boroughs.

EDA

Then performed exploratory data analysis on a sample of the cleaned dataset, to understand distribution and spread of the attributes. This also helps to remove outliers in the numerical features.

The continuous numeric fields (computed total charge, total amount, fare amount, tip, duration, trip distance) are skewed to the right. Some numeric fields are discrete (e.g., passenger count, congestion surcharge, airport fee, etc.).

Exploratory data analysis was conducted on the cleaned dataset to understand the distribution, spread, and quality of key attributes. Histograms and box plots were used to examine the continuous numerical variables (e.g., computed total charge, total amount, fare amount, tip amount, trip duration, and trip distance) which all exhibit right-skewness. The visualizations also helped identify and validate outlier removal decisions during later further cleaning. Several numerical fields, including passenger count, congestion surcharges, and airport fees, behave as discrete values.

Count plots were generated for categorical features such as payment type, borough, service zone, and rush hour indicator to assess whether category frequencies aligned with expectations.

Correlations among numerical features were examined using a correlation matrix to assess potential relationships between predictors and the target variable. As expected, the major fare component fields show strong positive correlations with the target `computed_total_charge`, since the target is directly derived as the sum of these components, and fare variables exhibit higher collinear relationships with each other. This analysis helps identify redundant features, understand multicollinearity, and assist feature selection for predictive modeling.

Scatter plots were used to further explore the relationships between key numerical features and the target variable, `computed_total_charge`. Trip distance shows a strong positive relationship with the total charge; trip duration also has a moderate positive correlation with total fares. Average speed exhibits only a mild positive relationship with the target.

Outlier Removal

After initial cleaning and exploratory analysis, there are still outliers in the dataset that might impact the training result of the model. With PySpark's `approxQuantile`, the 99.5 percentile threshold can be estimated for filtering out outliers. Since the important features are heavily right skewed, indicating extreme long duration, long distance, and high charge, the 0.5% on the upper tail are removed for reducing disproportionately large values that impairs model performance.

Prepare Final Training Dataset

The original dataset contains 10 months' amount of trip records, and the data volume would be too large for efficient computation and training an XGBoost regressor. A small random sample (2%) is drawn for model training containing about 440k records, which is sufficient to train an XGBoost regression model.

Categorical variables require transformation before being used in machine learning models. PySpark ML's `StringIndexer` handles translating nominal variables into values, and `OneHotEncoder` transforms these values into sparse one-hot encoded vectors. Numerical attributes are retained in their original formats, since XGBoost does not require normalization of numerical variables. All features are then combined into a single feature vector using `VectorAssembler`. The pipeline also extracts and stores metadata describing the feature mappings, enabling later interpretation of XGBoost feature importances.
The prepared data is then written as csv and saved to the GSC bucket.

VertexAI Custom Training

The training process was built using two distinct Python files stored on the local disk:

- task.py: The XGBoost regressor model training code. It reads the processed NYC Trip data, performs a 70/15/15 split for training/validation/testing, trains the XGBoost regression model, and calculates the final coefficient of determination score. Early stopping is applied to prevent the model from overfitting with large depth.

- Activation code: Uses Vertex AI SDK to trigger the Vertex AI API and launch the training code as a Custom Training Job. Compared to managing training applications through the Vertex AI interface, using Vertex AI SDK helps the process to be more manageable and easier for reproduction.

Since the model selected was XGBoost, and Vertex AI offers a pre-built container that manages the training environment, no custom Docker image or training application packaging is needed. The final trained model is saved in the required `.bst` format and automatically uploaded to the Vertex AI model directory using the environment variable `AIP_MODEL_DIR`, allowing integration with Vertex AI Model Registry.

The model was evaluated against a held-out test set to ensure the metrics were not biased by the training data. The Coefficient of Determination (R square) was used to assess how well the features explained the variance in taxi fares.

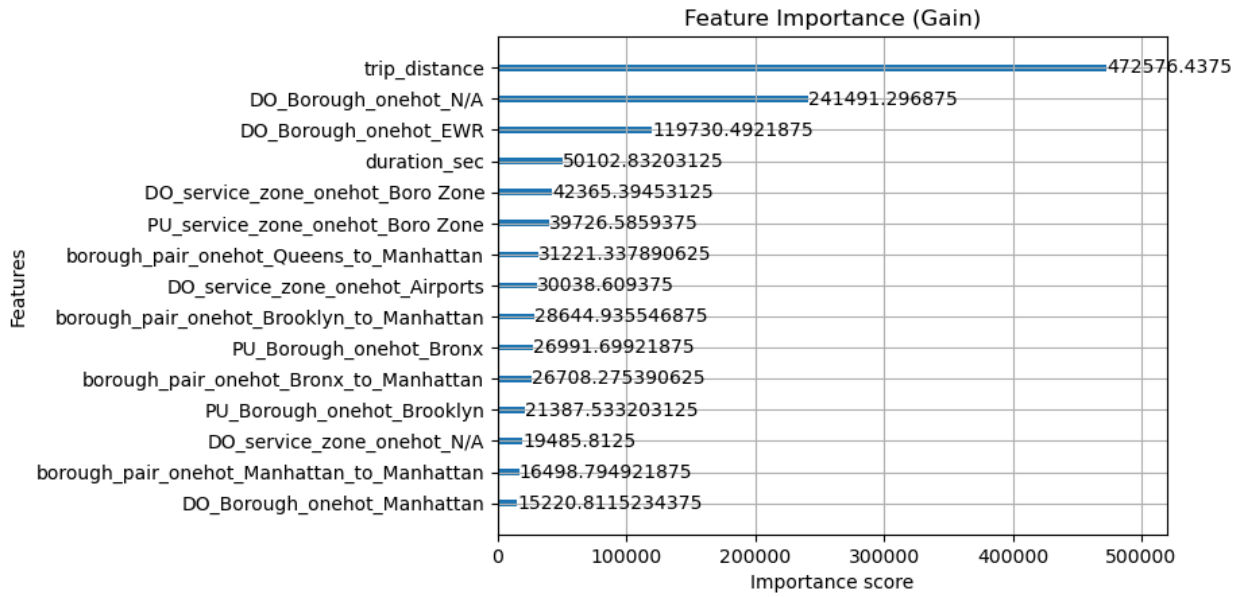
Model Performance & Inferences

Model performance can be observed with training logs. The model converges rapidly within the first 100 steps, with root mean square error reduced from 22.49 to l 5.39. The model stopped early at 278 with RMSE = 5.25, at this point, the reduction in error is no longer significant, and terminating further iteration prevents the model from over fitting.

```
[0]     validation_0-rmse:22.48975
[100]   validation_0-rmse:5.39397
[200]   validation_0-rmse:5.29056
[278]   validation_0-rmse:5.24533
```

The final Coefficient of Determination on the held-out test set is `Test R2 Score: 0.9495275021795804`. It indicates that the model's features account for 95% of the variance in the computed total charge, which is a very high performance for a regression model.

With the previously stored feature metadata, we can map the vectorized and assembled features back to original variables for better interpretation of feature importance. The feature importance analysis provides better insights into which features make the most contribution to the gain of the model.



The trip distance is the dominant feature, which aligns with the expectation that fares are charged majorly based on travel distance. The next two important features are drop off boroughs, N/A might infer places outside the five boroughs, and EWR refers to Newark airport. These features either suggest far trip distance or might be included in trips with tolls and airport fees, which are directly related to fares. Duration also appears to be one of the important features, as it is related to trip distance, and can also reflect congestion conditions.

Lessons Learned

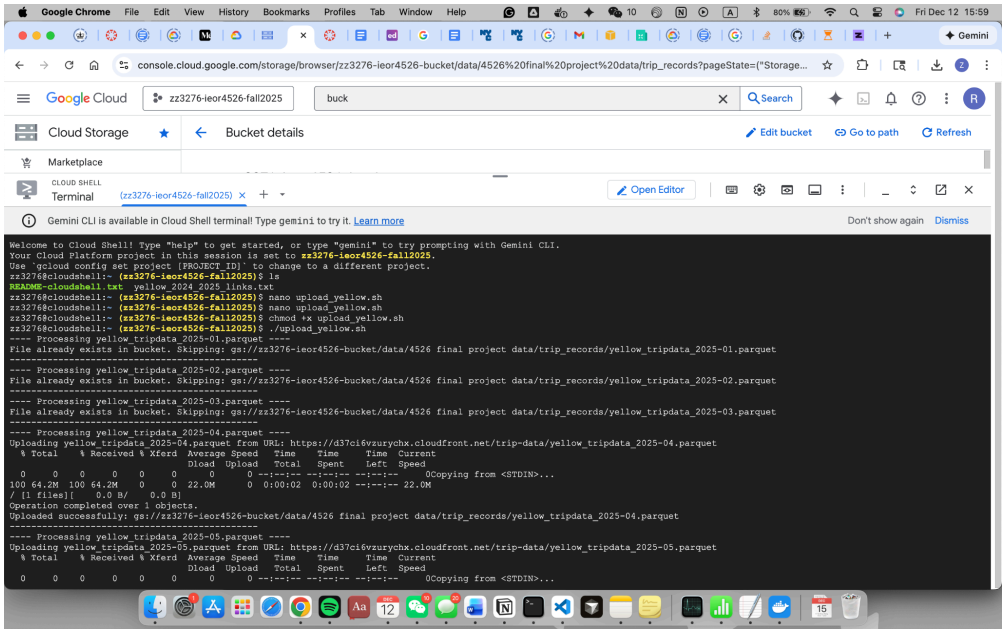
To process 10 months of Yellow Taxi trip records, which is a massive volume of data, processing with PySpark on a distributed cluster provides an efficient solution. This approach was significantly more efficient than Pandas, as Spark can handle large-scale datasets through parallelism and distributed computing. And running the transformations on clusters further accelerates the process.

While the NYC TLC dataset is provided in a well-structured format with correct data types and have clear schema and documentations, a critical lesson learned was that structural cleanliness does not equal logical accuracy. In realistic data, there is still much cleaning and preprocessing required to eliminate errors in data entry. Outlier detection relies on domain knowledge and practical heuristics, to assure the distribution of data aligns with expectation.

When exploring Vertex AI Custom Job, I understand it is a very powerful tool for building machine learning applications. The Custom Training task competed within 5 minutes and achieved high model performance. This project uses a pre-built container for Vertex AI Training, but building a customized docker container gives more control of the process and enables more customized training code. Furthermore, the current workflow is not automated enough, and to improve this, the job can be written as an application that orchestrates model training, inferences, and deployment.

Appendices

Using Google Shell terminal for downloading and writing parquet data in bucket



Training log screenshot